GPS Code Tracking Receiver

MATLAB/Simulink Implementation

Author: Alireza Ramyad
Tampere University
2026

System Implementation

I built a receiver with Early, Prompt, and Late correlators, followed by integrate-and dump accumulators. The DLL discriminator was implemented as a normalized noncoherent structure using $(E - L)/ (E + L)$.

Debugging Process and Issues

Problem 1: DLL output was NaN at startup

I suspected a division by zero problem. After checking the signals, we saw that (E + L) was sometimes zero. It got fixed by adding a small constant (1e-12) to the denominator.

Problem 2: DLL output was always zero

I suspected that the C/A code was not changing. After inspecting the generator, I found a wrong switch block forcing the output to constant 1. I removed the switch and restored the lookup table output.

Problem 3: Early, Prompt, and Late were identical

I suspected incorrect timing. After fixing the code generator, confirmed that Early, Prompt, and Late signals were different.

Problem 4: Output magnitude too large

I suspected wrong scaling. Initially I used incorrect gain. Then corrected the scaling to 0.5*Tc, which gave realistic values.

Problem 5: No change with SNR
I suspected that noise was not updating. So, checked the noise variance (Pn) and verified it changes with SNR. I also measured std(I_front) and confirmed noise increases when SNR decreases.

SNR and Integration Time

Initially, I expected the DLL output standard deviation to increase when SNR decreases. However, the results stayed almost constant (~84–85 m).

I suspected multiple issues:
- Noise not updating
- Wrong discriminator scaling
- Measurement error due to short simulation
tested each suspicion:

- Verified noise by checking std(I_front)
- Fixed scaling in discriminator
- Increased simulation time and removed startup samples

After verification, the system is concluded is correct. The flat result happens because the discriminator is normalized. This reduces sensitivity to amplitude changes caused by noise.

Phase and Frequency Errors

I expected phase error to affect DLL output. However, results stayed constant.

Also suspected the DLL might ignore phase. After reviewing the equations, I confirmed that the DLL uses $I^2 + Q^2$, so it is insensitive to phase rotation.

For frequency error, I expected degradation at high values (especially 1000 Hz). However, results remained similar. I suspected that frequency error effect is reduced due to:

- Short integration time
- Normalized discriminator
I confirmed that although frequency error should reduce correlation, the normalized structure limits its visible effect in our measurements.

Evaluation of Discriminator Outputs:

Item	*SNR*	*PIT (s)*	*DLL output error std dev (m)*
Nominal case	-18 dB	0.001	84.25 m
Factor of 2	-21 dB	0.001	84.22 m
Factor of 2, with PIT adjusted to get performance equivalent to nominal case	-21 dB	0.002	85.16 m
Factor of 4	-24 dB	0.001	84.19 m
Factor of 4, with PIT adjusted to get performance equivalent to nominal case	-24 dB	0.004	85.01 m
Factor of 10	-28 dB	0.001	84.17 m
Factor of 10, with PIT adjusted to get performance equivalent to nominal case	-28 dB	0.010	82.93 m

Evaluation of Discriminator Outputs in Presence of Phase and Frequency Error:

Item	*Phase Error (deg)*	*Frequency Error (Hz)*	*DLL output error std dev (m)*
No errors	0	0	84.30
30 deg phase error, perfect frequency	30	0	84.30
220 deg phase error, perfect frequency	220	0	84.30
No phase error, 50 Hz frequency error	0	50	83.67
No phase error, 250 Hz frequency error	0	250	84.76
No phase error, 500 Hz frequency error	0	500	84.20
90 deg phase error, 500 Hz frequency error	90	500	84.20
-74 deg phase error, 500 Hz frequency error	-74	500	84.20
No phase error, 750 Hz frequency error	0	750	85.16
No phase error, 1000 Hz frequency error	0	1000	84.42